

■ Python Programming

Complete Exam Reviewer

Syntax · OOP · Encapsulation · Inheritance · Polymorphism

Part 1 Python Basics

Variables · Data Types · Control Flow · Loops

Part 2 OOP & Encapsulation

Classes · Objects · Access Modifiers · Getters/Setters

Part 3 Inheritance & Polymorphism

Types of Inheritance · super() · Overriding · Duck Typing

Part 1 — Python Basic Syntax

■ Key Definitions

Syntax	The set of rules that dictate how to write code in a programming language — like grammar for code.
Variable	A named storage location in memory that holds a value. Think of it as a labeled box that stores data.
Data Type	Defines what kind of value a variable holds (e.g., text, number, True/False) and what you can do with it.
Control Flow	The order in which code is executed. It can be controlled using conditions and loops.
Loop	A block of code that repeats automatically until a condition is met or a sequence is finished.
Casting	Converting a value from one data type to another (e.g., turning the number 3 into the string '3').
Global Var	A variable declared outside all functions — accessible from anywhere in the program.
Local Var	A variable declared inside a function — only exists and works within that function.

■ Data Types & Collections

Type	Syntax	Ordered?	Changeable?	Duplicates?
List	[]	Yes	Yes	Yes
Tuple	()	Yes	No	Yes
Set	{ }	No	Add/Remove	No
Dictionary	{k:v}	No	Yes	Keys unique

■ Arithmetic Operators

Operator	Symbol	Example	Result
Addition	+	5 + 3	8

Subtraction	-	5 - 3	2
Multiplication	*	5 * 3	15
Division	/	10 / 4	2.5
Modulus	%	10 % 3	1 (remainder)
Floor Division	//	10 // 3	3 (rounded down)
Exponent	**	2 ** 3	8 (power)

■ Control Flow — If Statements

if	Runs code ONLY if the condition is True. If False, skips the block entirely.
if-else	Runs one block if True, a different block if False — like a fork in the road.
if-elif-else	Checks multiple conditions in order. Runs the first block that is True; else is the fallback.
Nested if	An if statement inside another if. Used for conditions within conditions.
not	Inverts the condition — True becomes False, False becomes True.
and	Both conditions must be True for the block to run.
or	At least one condition must be True for the block to run.
in	Checks if an item EXISTS inside a list, tuple, or string.

■ Loops

while loop	Repeats code as long as the condition is True. Stops when the condition becomes False.
for loop	Iterates (goes through) every item in a sequence — list, tuple, string, or range.
range()	Generates a sequence of numbers. range(5) gives 0, 1, 2, 3, 4.
break	Immediately stops the loop, no matter what the condition is.
else (loop)	Runs once after the loop finishes normally (not interrupted by break).
Nested loop	A loop inside another loop. The inner loop runs fully for every step of the outer loop.

Part 2 — OOP & Encapsulation

■ Key Definitions

OOP	Object-Oriented Programming — a style of coding that models real-world things as objects with data and behavior.
Object	A programming representation of a real-world thing (person, car, animal). It has state (data) and behavior (actions).
State	The data/attributes stored inside an object (e.g., a dog's name, color, breed).
Behavior	The actions an object can perform — defined as methods (e.g., a dog can bark, run, fetch).
Class	A blueprint or template used to create objects. It defines what attributes and methods objects will have.
Instance	A specific object created from a class. If 'Person' is the class, 'James' is an instance.
Attribute	A data value stored inside an object (e.g., name = 'Alice', age = 20).
Method	A function that belongs to a class and defines what an object can do.
<code>__init__</code>	A special method (constructor) that runs automatically when a new object is created. Used to set initial values.
<code>self</code>	Refers to the specific instance of the class. It lets you access that object's own attributes and methods.

■ Variable & Method Types

Type	Declared Where	Shared?	Use Case
Instance Variable	Inside <code>__init__</code>	No — unique per object	Store object-specific data
Class Variable	Outside methods	Yes — shared by all	Data common to all objects
Instance Method	Inside class	Accesses instance + class data	Most common methods
Class Method	<code>@classmethod</code>	Class data only	Factory methods, class-level ops

Static Method	@staticmethod	Neither	Utility/helper functions
---------------	---------------	---------	--------------------------

■ Encapsulation & Access Modifiers

Encapsulation	Bundling data (variables) and the methods that work on them together inside one class.
Data Hiding	Restricting direct access to an object's internal data to protect it from accidental changes.
Public member	Default. Accessible from anywhere — inside or outside the class. No prefix needed.
Protected (_)	Prefix with ONE underscore. Accessible inside the class AND its subclasses (for inheritance).
Private (__)	Prefix with TWO underscores. Accessible ONLY within the class. Strongest restriction.
Getter	A method that reads (gets) the value of a private attribute safely.
Setter	A method that changes (sets) the value of a private attribute, usually with validation.
Name Mangling	A way to access private members from outside using <code>__ClassName__variableName</code> .

Part 3 — Inheritance & Polymorphism

■ Key Definitions

Inheritance	A mechanism where a child class automatically gets the attributes and methods of a parent class — avoids rewriting code.
Parent Class	Also called Base Class. The original class whose properties are inherited by others.
Child Class	Also called Subclass. The class that inherits from a parent. It can also add its own features.
Polymorphism	Means 'many forms.' The same method or function behaves differently depending on the object that calls it.
super()	A built-in function that lets you call a method from the parent class inside the child class.
issubclass()	A built-in function that checks if one class is a child of another. Returns True or False.
Method Override	When a child class redefines a method from the parent class with the same name but different behavior.
MRO	Method Resolution Order — the sequence Python follows to find a method: current class first, then parents left to right.
Duck Typing	A concept where Python cares about what an object CAN DO, not what type it IS. 'If it acts like a duck, it is a duck.'
Overloading	Defining methods with the same name but different parameters. Python does NOT support true overloading — only the last definition works.
Op. Overloading	Changing how operators like + or * behave for custom objects using special methods like <code>__add__</code> .

■ Types of Inheritance

Type	Description	Example Pattern
Single	One parent → one child	<code>class B(A):</code>
Multiple	One child → multiple parents	<code>class C(A, B):</code>

Multilevel	Chain: $A \rightarrow B \rightarrow C$ (grandparent/parent)	class C(B): where B(A)
Hierarchical	One parent $\rightarrow$ many children	class B(A): / class C(A):
Hybrid	Combination of two or more types	Mix of above

■ Polymorphism Methods

Method Override	Child class redefines a parent's method. Same name, different behavior. Most common form of polymorphism.
Poly w/ Functions	A function that accepts any object and calls a method on it — works for different classes as long as they have that method.
len() example	len('hello') counts characters; len([1,2,3]) counts items. Same function, different outputs = polymorphism.
Duck Typing	No need to check what type an object is. If it has the method you need, just call it.
Op. Overloading	Redefine operators for your class using methods like <code>__add__</code> , <code>__sub__</code> , etc.

■ Master Cheat Sheet — All Topics

Quick Reference: All Key Terms

Term	One-Line Definition
Syntax	Rules for writing valid Python code
Variable	Named box in memory that stores a value
Data Type	Kind of value a variable holds (str, int, float, bool)
List []	Ordered, changeable, allows duplicates
Tuple ()	Ordered, unchangeable (read-only), allows duplicates
Set {}	Unordered, no duplicates, partial changes
Dictionary {k:v}	Key-value pairs; unordered but indexed by key
if / elif / else	Run different blocks based on conditions
while loop	Repeat while condition is True
for loop	Iterate through a sequence
break	Stop the loop immediately
OOP	Programming style using objects to model real things
Class	Blueprint for creating objects
Object/Instance	A specific item made from a class blueprint
__init__	Constructor – runs when an object is created
self	Refers to the current object inside a class
Encapsulation	Bundling data + methods into one class
Data Hiding	Restricting access to internal data
Public	Accessible everywhere (no prefix)
Protected _	Accessible in class + subclasses
Private __	Accessible only inside the class
Getter	Method to READ a private attribute
Setter	Method to WRITE/UPDATE a private attribute
Inheritance	Child class gets features of parent class
super()	Call parent class method from child

issubclass()	Check if a class is a child of another
Method Override	Child redefines parent's method
MRO	Order Python searches for a method
Polymorphism	Same method, different behavior per object
Duck Typing	Behavior matters, not type
Op. Overloading	Redefine how + - * work for your class